

Special Participation E 2

Edward Zhang

December 2025

1 Introduction

I am taking EE 226A this year, which mentioned the concept of Kalman Filter as following:

We also mentioned the concept of Kalman Filter between we are introducing

4.5.1 • Kalman prediction

The prediction version of the Kalman filter is given as follows:

Initialize: $\hat{X}_{0|-1} = 0$, $\sigma_{0|-1}^2 = P_0$.

For $i = 0, 1, 2, \dots$ **do:**

$$\hat{X}_{i+1|i} = a_i \hat{X}_{i|i-1} + k_i (Y_i - \hat{X}_{i|i-1}) \quad (4.10)$$

where

$$k_i = a_i \sigma_{i|i-1}^2 (\sigma_{i|i-1}^2 + N_i)^{-1} \quad (4.11)$$

is the *Kalman gain*, and the variance of the prediction error $\sigma_{i+1|i}^2 := \text{Var}(X_{i+1} - \hat{X}_{i+1|i})$ is given by

$$\sigma_{i+1|i}^2 = a_i (a_i - k_i) \sigma_{i|i-1}^2 + Q_i. \quad (4.12)$$

Example 4.12. Let all $a_i = 1$, $Q_i = 0$, $N_i = \sigma_Z^2$, and $P_0 = \sigma_X^2$. In this case, the observations are given by $Y_i = X + Z_i$, $i \geq 0$, and the Kalman filter equations simplify to the scenario considered in Example 4.9. ■

Theorem 4.13. *The Kalman prediction algorithm stated above satisfies $\hat{X}_{i+1|i} = \mathbb{E}[X_{i+1}|Y^i]$, and $\sigma_{i+1|i}^2$ is equal to the variance of the corresponding prediction error.*

Figure 1: Caption

RNNs. Therefore, I want to relate both concepts and discover the relationship and connection between those with CharGPT 5.1 thinking version.

2 Conclusion

In this chat I clarified how Kalman filters relate to RNNs and what each symbol means. I learned that a (linear) Kalman filter can be seen as a very special linear RNN: the latent state X_i in the state-space model is like the true underlying process, the observation Y_i corresponds to the RNN input x_i , and the Kalman estimate $\hat{X}_{i|i}$ (plus its variance) plays the role of the hidden state h_i . In RNN notation we usually don't write an explicit X_i ; instead, the hidden state is supposed to encode our best internal representation of that latent state. I also sorted out the "output" y_i : in Kalman terms I can think of y_i as just a readout of the filter's belief, e.g. $y_i = \hat{X}_{i|i}$ or the predicted observation, and even though we write $y_i = g(h_i)$, it still depends on the current input x_i indirectly through the update $h_i = f(h_{i-1}, x_i)$. Overall, I now see Kalman filtering as a model-based, optimal linear RNN for linear-Gaussian systems, while a generic RNN is a learned, more flexible version of a state estimator/filter.

Kalman Filter and RNN Comparison

User

[File]: [file_000000002e18722fa8c4843d23beb175-sanitized.png](#)

[File]: [file_00000000b92071f58e0c9189105dd0bf-sanitized.png](#)

The images are from another class's notes. What is the connection between Kalman Filter/Kalman prediction with RNN?

ChatGPT

Short version: a (linear) Kalman filter **is literally a very special RNN** whose hidden state is “belief about the latent state (mean + uncertainty)” and whose weights are fixed by a probabilistic model instead of learned from data.

1. Put the Kalman predictor in RNN form

Your notes have the 1-D state-space model

$$\begin{aligned} \backslash[\\ X_{i+1} &= a_i X_i + U_i, \quad Y_i = X_i + V_i \\ \backslash] \end{aligned}$$

and the (prediction) Kalman update

$$\begin{aligned} \backslash[\\ \hat{X}_{i+1|i} \\ = a_i \hat{X}_{i|i-1} + k_i (Y_i - \hat{X}_{i|i-1}), \\ \backslash] \end{aligned}$$

plus a recursion for the variance $(\sigma^2_{i+1|i})$.

Now compare with a generic RNN cell (say, a simple Elman RNN):

$$\begin{aligned} \backslash[\\ h_{t+1} &= f(W_h h_t + W_x x_t + b), \\ \backslash] \end{aligned}$$

where (h_t) is the hidden state and (x_t) is the input.

Take

- hidden state $(h_i := \hat{X}_{i|i-1})$ (or the pair $(\hat{X}_{i|i-1}, \sigma^2_{i|i-1})$) if you track uncertainty),
- input $(x_i := Y_i)$,
- linear activation $(f(x)=x)$.

Then the Kalman predictor step is exactly

$$\begin{aligned} \backslash[\\ h_{i+1} &= a_i h_i + k_i (x_i - h_i) \\ &= (a_i - k_i) h_i + k_i x_i, \\ \backslash] \end{aligned}$$

which is of the form $(h_{i+1} = W_h h_i + W_x x_i)$ with

$$\begin{aligned} \backslash[\\ W_h &= a_i - k_i, \quad W_x = k_i. \\ \backslash] \end{aligned}$$

So: **a scalar Kalman filter is a 1-layer linear RNN with a 1-dimensional hidden state,** in

higher dimensions it's a linear RNN with matrix weights.

The extra variance recursion is just an additional hidden state dimension encoding your **uncertainty**, also updated recurrently.

2. Conceptual connection

Both Kalman filters and RNNs:

- Process a **sequence** of observations $(Y_0, Y_1, \dots)$.
- Maintain a **latent state** that summarizes all past information.
- Update this state **recursively** using the previous state and the new observation (Markov / causal).

Differences:

- In a (classical) Kalman filter, the update form and "weights" (the Kalman gain (k_i) , the transition (a_i) , etc.) are derived analytically from a **linear-Gaussian probabilistic model** and noise variances (Q_i, N_i) . No learning from data is required: it's the **optimal linear estimator** for that model.
- In a standard RNN, the update rule is chosen (often non-linear, with tanh/ReLU) and the weights (W_h, W_x) are **learned from data** via backpropagation, not derived from assumed noise statistics. It can approximate much more general dynamics but loses exact optimality guarantees.

You can view a general RNN as doing **approximate Bayesian filtering** in some unknown latent state-space model. The linear Kalman filter is the special case where:

- the model is linear,
- noise is Gaussian,
- the RNN is linear,
- and the weights are those that minimize MSE given the known model.

3. Why your classes mention both

So likely:

- The other class (with these notes) is teaching **state-space models and optimal filtering** (Kalman).
- Your RNN / deep learning class is teaching that we can **replace the hand-derived Kalman update with a learned recurrent update**, or combine them (e.g., "KalmanNet", "neural Kalman filters") where an RNN learns parts of the model or the gain.

Big picture: **Kalman filter = model-based, linear-Gaussian RNN with interpretable parameters and uncertainty; generic RNN = data-driven, potentially nonlinear Kalman-like filter with learned parameters.**

User

What does X_i represent in RNN? I understand the what hidden state represents in Kalman updates.

ChatGPT

Nice, that's a really good thing to tease apart.

In the **Kalman** notation you pasted we have three different "kinds" of variables:

- **True latent state:** (X_i)
- **Observation:** (Y_i)
- **Estimator's internal state:** $(\hat{X}_{i|i-1}, \hat{X}_{i|i})$ (the filter's belief about (X_i))

In a *vanilla RNN* we normally only write two sequences:

- **Input:** x_i
- **Hidden state:** h_i (and maybe an output o_i)

So there is **no direct analogue of X_i** as a separate symbol in standard RNN notation. Conceptually, though, you can line them up like this:

- Kalman: **true world state** X_i
- Kalman filter: **internal estimate of that state** $\hat{X}_{i|mid i}$
- RNN: **learned representation / internal memory** h_i

If we force the analogy:

- $Y_i \rightarrow x_i$ (the sequence of observed inputs)
- $\hat{X}_{i|mid i} \rightarrow h_i$ (the RNN's hidden state, a learned estimate/encoding of whatever underlying process produced the data)
- The "ideal" X_i in Kalman is like the **unknown, true latent state of the world** which the RNN **tries** to implicitly represent in h_i , but we never write X_i explicitly in the RNN equations.

Example intuition:

- In tracking: X_i = true position/velocity of an object, Y_i = noisy sensor reading, $\hat{X}_{i|mid i}$ = Kalman's best guess.
- In an RNN version, you just see noisy inputs x_i and maintain a hidden state h_i that the network learns so that it can, say, predict the next measurement. Conceptually h_i is playing the role of an **estimated** X_i .

So: **in RNN land we don't usually have a separate variable named X_i** ; the thing corresponding to the Kalman X_i is the (possibly unknown) underlying process, and the thing corresponding to the Kalman **estimate** $\hat{X}_{i|mid i}$ is the RNN's hidden state h_i .

User

In RNN (assume single layer), we also obtain output y_i from each block in addition to hidden state h_{i+1} , right? What does y_i represent in Kalman filter notation then?

ChatGPT

Right – in a "textbook" single-layer RNN cell you often see something like

$$\begin{aligned} h_{i+1} &= f(W_h h_i + W_x x_i), \\ y_i &= g(h_i), \end{aligned}$$

so there is an **input** x_i , a **hidden state** h_i , and an explicit **output** y_i .

In the scalar Kalman predictor from your notes we have

- latent state: X_i
- observation (what we actually see): Y_i
- filter's internal belief: $\hat{X}_{i|mid i-1}$, $\hat{X}_{i|mid i}$
- plus the variance $\sigma^2_{i|mid i-1}$, $\dots$

There is **no extra symbol for "output"** in the Kalman notation; the "output" of the algorithm is just whatever quantity we care about (usually the state estimate).

So, when you map to the RNN view, you usually do something like:

- RNN input:

$$\begin{aligned} &x_i \rightarrow Y_i \\ &\quad \text{(measurement/observation)} \end{aligned}$$

```

\]
- RNN hidden state:
\[
h_i \rightarrow (\hat{X}_{i|i-1}, \sigma^2_{i|i-1})
\quad \text{(belief about latent state + uncertainty)}
\]
- RNN output:
pick a function of that belief, e.g.
- **state estimate:**  $y_i := \hat{X}_{i|i}$ , or
- **predicted observation:**  $y_i := \hat{Y}_i := \mathbb{E}[Y_i | Y^{i-1}] = \hat{X}_{i|i-1}$  in your simple model.

```

So:

- In Kalman-filter notation, y_i *doesn't exist as a separate variable*.
- If you want to align the two worlds, you *define* a y_i as some readout of the Kalman filter's state, typically the current best estimate $\hat{X}_{i|i}$ or the predicted observation $\hat{Y}_i$.

That's exactly what an RNN does: $y_i = g(h_i)$ is just a readout of its internal state, same as taking $\hat{X}_{i|i}$ or $\mathbb{E}[\hat{X}_{i|i}]$ as the "output" of a Kalman filter.

User

Why does y_i depend only in h_i instead of both h_i and x_i ? If we comprehend y_i as $X_{i|i}$ = $L[X_i | Y^i]$, it should depends on Y_i (x_i) and $L[X_i | Y^{i-1}]$ (h_i), right?

ChatGPT

Yeah, good catch - this is one of those "indexing / notation" traps.

There are two separate points:

1. Even in the standard RNN, y_i **does** depend on x_i

Textbook RNN:

```

\[
h_i = f(W_h h_{i-1} + W_x x_i), \quad
y_i = g(W_y h_i).
\]

```

If you substitute the first equation into the second,

```

\[
y_i = g(W_y f(W_h h_{i-1} + W_x x_i)),
\]

```

so mathematically y_i is a function of **both** h_{i-1} and x_i . We just *factor* that dependence through the hidden state h_i .

So saying " y_i depends only on h_i " really means "we choose to *represent* the dependence on the entire history $x_{1:i}$ via a compressed state h_i ".

2. Matching this to the Kalman filter

In the Kalman filter:

- prior (prediction): $\hat{X}_{i|i-1} = \mathbb{E}[X_i | Y^{i-1}]$
- posterior (filtered): $\hat{X}_{i|i} = \mathbb{E}[X_i | Y^i]$

and the update is

$$\begin{aligned} & \hat{X}_{i|i} \\ &= \hat{X}_{i|i-1} + k_i(Y_i - \hat{X}_{i|i-1}), \end{aligned}$$

which clearly depends on both the **previous belief** $(\hat{X}_{i|i-1})$ and the new observation (Y_i) .

To put this in RNN form, do:

- input: $(x_i := Y_i)$
- hidden state: $(h_{i-1} := \hat{X}_{i|i-1})$ (prior at time (i))
- new hidden state:

$$\begin{aligned} & h_i := \hat{X}_{i|i} \\ &= f(h_{i-1}, x_i) \\ &= h_{i-1} + k_i(x_i - h_{i-1}) \end{aligned}$$
- output: $(y_i := h_i)$ (or $(y_i := W_y h_i)$ if you want a linear readout)

Now your interpretation is exactly right:

$$y_i = h_i = \mathbb{E}[X_i \mid Y^i]$$

and this *does* depend on both (Y_i) and the previous estimate $(\mathbb{E}[X_i \mid Y^{i-1}])$; that dependence is just “hidden inside” the update that produced (h_i) .

So, in words:

(y_i) is written as a function of (h_i) only because (h_i) already encodes all the information from both the past and the current input. When we set $(h_i = \mathbb{E}[X_i \mid Y^i])$ in the Kalman analogy, $(y_i = g(h_i))$ is just a readout of that filtered estimate, and its dependence on (Y_i) runs through the update $(h_i = f(h_{i-1}, x_i))$.

If you really wanted, you *could* define an RNN with $(y_i = g(h_i, x_i))$; it's just redundant, because any dependence on (x_i) can be pushed into the definition of (h_i) .

Exit rates and embedded chain

User

[File]: [file_00000000bb5471fba4d36f1871af20ad-sanitized.png](#)

ChatGPT

```
from PIL import Image, ImageOps, ImageFilter, ImageEnhance, ImageStat, ImageChops,
ImageFilter
img = Image.open('/mnt/data/e46f3b08-e81f-422d-b674-155591d7e4dc.png')
display(img)
```

ChatGPT

```
import numpy as np, math

def p_i(i):
    return 0.25 * (3/4)**i
```